MOLLI: Model Optimization via Likelihood-based Introgression

Mathematical Details and Worked Examples

Course Notes for the Bayesian Gene Scoring Procedure

Contents

1 Mathematical Preliminaries **2**

1.1 Random Variables and Variance . . . 2

1.2 The Normal (Gaussian) Distribution . . . 2

1.3 Bayesian Inference: The Core Idea . . . 3

1.4 Conjugate Normal Model . . . 3

1.5 Gradients and Partial Derivatives . . . 4

1.6 First-Order Taylor Approximation . . . 4

1.7 Perplexity . . . 4

2 The MOLLI Algorithm **5**

2.1 Setup: Genes and Dual-Strand Encoding . . . 5

2.2 The Evolution Cycle . . . 5

3 Gene Scoring via Gradient Approximation **6**

3.1 The Question We Want to Answer . . . 6

3.2 The Taylor Approximation . . . 6

4 Split-Half Noise Estimation **7**

4.1 Why We Need Noise Estimation . . . 7

4.2 The Split-Half Procedure . . . 7

5 Empirical Bayes Shrinkage **8**

5.1 The Generative Model . . . 8

5.2 Estimating the Parameters (Method of Moments) . . . 8

5.3 Computing the Posterior . . . 9

5.4 Computing Posterior Probabilities . . . 9

6 The Repair Decision **10**

6.1 Multi-Objective Aggregation 10
6.2 The Trade-off Score 10
6.3 Decision Rules 10

7 Experimental Results **11**

8 Summary **12**

1 Mathematical Preliminaries

Before describing the MOLLI algorithm, we review several concepts from probability and calculus that the method relies on. Readers comfortable with Bayesian statistics and gradients may skip to Section 2.

1.1 Random Variables and Variance

A **random variable** X is a quantity whose value is determined by a random process. For example, if we roll a die, the outcome X could be any integer from 1 to 6.

The **expected value** (or mean) of X is the average value we would observe over many repetitions:

$$\mathbb{E}[X] = \mu = \sum_x x \cdot P(X = x)$$

For continuous variables, the sum becomes an integral.

The **variance** measures how spread out the values of X are around the mean:

$$\mathrm{Var}(X) = \mathbb{E}\big[(X - \mu)^2\big] = \mathbb{E}[X^2] - \mu^2$$

A small variance means X tends to be close to μ; a large variance means X is often far from μ. The **standard deviation** is $\sigma = \sqrt{\mathrm{Var}(X)}$, which has the same units as X.

Example

If X represents the height of randomly chosen adults (in cm), we might have $\mu = 170$ and $\sigma = 10$. This means most people are within about 10 cm of 170 cm.

1.2 The Normal (Gaussian) Distribution

The **normal distribution** $\mathcal{N}(\mu, \sigma^2)$ is a bell-shaped distribution parameterized by its mean μ and variance σ^2. Its probability density function is:

$$f(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$$

Key properties:

- About 68% of values fall within $\pm 1\sigma$ of the mean.
- About 95% fall within $\pm 2\sigma$.
- The **cumulative distribution function** (CDF), written $\Phi(z)$ for the standard normal $\mathcal{N}(0,1)$, gives the probability that a draw is $\leq z$:

$$\Phi(z) = P(Z \leq z) = \int_{-\infty}^{z} \frac{1}{\sqrt{2\pi}} e^{-t^2/2}\, dt$$

- Therefore, $P(Z > z) = 1 - \Phi(z)$.

Example

If $Z \sim \mathcal{N}(0,1)$, then $P(Z > 0) = 1 - \Phi(0) = 0.5$ (by symmetry). If $Z \sim \mathcal{N}(2,1)$, then $P(Z > 0) = 1 - \Phi(-2) \approx 0.977$—we are 97.7% confident Z is positive.

1.3 Bayesian Inference: The Core Idea

In **Bayesian inference**, we start with a *prior belief* about some unknown quantity θ, observe data D, and update our belief to get a *posterior*:

$$\underbrace{P(\theta \mid D)}_{\text{posterior}} \propto \underbrace{P(D \mid \theta)}_{\text{likelihood}} \times \underbrace{P(\theta)}_{\text{prior}}$$

In words: what we believe after seeing the data is proportional to how likely the data would be under each possible value of θ, weighted by what we believed before.

Key Idea

Bayesian inference lets us combine uncertain measurements with prior knowledge to get better estimates. When data is noisy, the prior "regularizes" our estimate, pulling it away from extreme values that are likely due to noise.

1.4 Conjugate Normal Model

A particularly clean case arises when both the prior and the likelihood are normal distributions. Suppose:

$$\text{Prior:} \qquad \theta \sim \mathcal{N}(\mu_0, \tau^2) \tag{1}$$

$$\text{Likelihood:} \qquad X \mid \theta \sim \mathcal{N}(\theta, \sigma^2) \tag{2}$$

Then the posterior is also normal:

$$\theta \mid X \sim \mathcal{N}\Big(B \cdot X + (1 - B) \cdot \mu_0, \;\; B \cdot \sigma^2\Big) \tag{3}$$

where the **shrinkage factor** is:

$$B = \frac{\tau^2}{\tau^2 + \sigma^2} \tag{4}$$

Key Idea

The posterior mean is a **weighted average** of the observed data X and the prior mean μ_0. The weight B depends on the **signal-to-noise ratio**: when $\tau^2 \gg \sigma^2$ (signal dominates), $B \approx 1$ and we trust the data. When $\sigma^2 \gg \tau^2$ (noise dominates), $B \approx 0$ and we trust the prior.

Example

Suppose we're estimating a baseball player's true batting average. Our prior is that most players bat around $\mu_0 = 0.260$ with $\tau = 0.03$. We observe a player batting $X = 0.400$ over 10 at-bats, with observation noise $\sigma = 0.15$. Then:

$$B = \frac{0.03^2}{0.03^2 + 0.15^2} = \frac{0.0009}{0.0234} \approx 0.038$$

Posterior mean $= 0.038 \times 0.400 + 0.962 \times 0.260 \approx 0.265$.
The small sample gives high noise ($\sigma \gg \tau$), so $B \approx 0$ and we barely trust the observation—our estimate is pulled almost entirely back to the league average. This is *shrinkage* at work.

1.5 Gradients and Partial Derivatives

A **gradient** $\nabla_\theta f$ is the vector of partial derivatives of a function f with respect to each element of a vector θ:

$$\nabla_\theta f = \left(\frac{\partial f}{\partial \theta_1}, \frac{\partial f}{\partial \theta_2}, \ldots, \frac{\partial f}{\partial \theta_d} \right)$$

Each partial derivative $\frac{\partial f}{\partial \theta_k}$ tells us how much f changes when we change θ_k by a tiny amount, holding all other parameters fixed.

1.6 First-Order Taylor Approximation

For a small change $\Delta\theta$ in the parameter vector, the change in f is approximately:

$$f(\theta + \Delta\theta) - f(\theta) \;\approx\; \sum_{k=1}^{d} \frac{\partial f}{\partial \theta_k} \cdot \Delta\theta_k \;=\; \nabla_\theta f \cdot \Delta\theta \tag{5}$$

This linear approximation works well when $\Delta\theta$ is small relative to the scale of θ. It avoids computing f at the perturbed point—we only need the gradient at the original point and the perturbation vector.

1.7 Perplexity

Perplexity (PPL) measures how well a language model predicts text. It is defined as:

$$\text{PPL} = \exp\left(\frac{1}{N} \sum_{i=1}^{N} -\log P(x_i \mid x_1, \ldots, x_{i-1}) \right)$$

where $x_1, \ldots, x_N$ are the tokens in a text and $P(x_i \mid \cdots)$ is the model's predicted probability for token x_i. Intuitively:

- PPL = 1: the model predicts every token with certainty.
- PPL = 10: on average, the model is as uncertain as choosing from 10 equally likely options.
- PPL = 50,000: the model is essentially guessing randomly.

Lower perplexity is better. The geometric mean (GM) of perplexities across domains gives a single summary of overall model quality.

2 The MOLLI Algorithm

2.1 Setup: Genes and Dual-Strand Encoding

A neural network has d parameters (weights) $\theta = (\theta_1, \theta_2, \ldots, \theta_d)$. We partition these into L non-overlapping groups called **genes**:

$$\theta = (g_1, g_2, \ldots, g_L), \qquad g_i = (\theta_{s_i}, \theta_{s_i+1}, \ldots, \theta_{e_i})$$

where $[s_i, e_i]$ is the index range of gene i. Each gene corresponds to a functional unit of the network: an attention head, an MLP block, or a normalization layer. For a 7-billion-parameter model, this gives $L = 928$ genes.

We maintain two copies of all parameters:

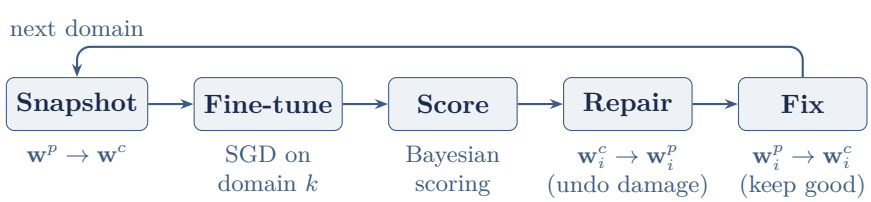
- **Primary strand $\mathbf{w}^p$**: the live, trainable weights (full precision).
- **Complement strand $\mathbf{w}^c$**: a quantized snapshot (int16) of the weights taken before training on the new domain. This represents the "healthy" state.

Biological Analogy

In the Amazon Molly, the two chromosome copies serve the same role: one may acquire a harmful mutation during reproduction, and the other provides a "backup" to repair it. In MOLLI, fine-tuning introduces "mutations" (weight changes), and the complement strand is the backup.

2.2 The Evolution Cycle

For each new domain k in the sequence, MOLLI executes five steps:



Step 1: Snapshot. Quantize the current primary weights into the complement strand: $\mathbf{w}^c \leftarrow \text{quantize}(\mathbf{w}^p)$. After this, $\mathbf{w}^c$ represents the pre-training state.

Step 2: Fine-tune. Train the primary weights $\mathbf{w}^p$ on the new domain k using standard stochastic gradient descent. This changes $\mathbf{w}^p$ but leaves $\mathbf{w}^c$ unchanged. The model now performs well on domain k but may have lost performance on prior domains.

Step 3: Score. For each gene i and each prior domain j, estimate how much damage (or benefit) the fine-tuning caused. This is the core mathematical contribution and is detailed in Sections 3–5.

Step 4: Repair. Genes identified as damaged are reverted by copying from the complement: $\mathbf{w}_i^p \leftarrow \mathbf{w}_i^c$. See Section 6.

Step 5: Fix. Genes identified as beneficial and harmless to all prior domains have their complement updated: $\mathbf{w}_i^c \leftarrow \mathbf{w}_i^p$, permanently incorporating the adaptation.

3 Gene Scoring via Gradient Approximation

3.1 The Question We Want to Answer

After fine-tuning on domain k, gene i's weights have changed from $\mathbf{w}_i^c$ (pre-training) to $\mathbf{w}_i^p$ (post-training). For each prior domain j, we want to know:

> *If we reverted gene i back to its pre-training state (replacing $\mathbf{w}_i^p$ with $\mathbf{w}_i^c$), how would the loss on domain j change?*

Let $\mathcal{L}_j(\theta)$ denote the loss on domain j's evaluation data when the model has parameters θ. After fine-tuning, the current parameters are $\mathbf{w}^p$. If we revert gene i, we get a **chimeric** parameter vector $\tilde{\theta}$ that is identical to $\mathbf{w}^p$ except for gene i:

$$\tilde{\theta}_k = \begin{cases} w_k^c & \text{if } k \in g_i \\ w_k^p & \text{otherwise} \end{cases}$$

The exact reversion effect is $\mathcal{L}_j(\tilde{\theta}) - \mathcal{L}_j(\mathbf{w}^p)$, but computing this requires a separate forward pass for each of the $L = 928$ genes and each of the J prior domains—$928 \times J$ forward passes total.

3.2 The Taylor Approximation

Instead, we use the first-order Taylor approximation (Eq. 5). The change from $\mathbf{w}^p$ to $\tilde{\theta}$ only affects the parameters in gene i, and the change is $\Delta\theta_k = w_k^c - w_k^p$ for $k \in g_i$ (zero elsewhere). Therefore:

$$\boxed{\delta_{ij} \;=\; \sum_{k \in g_i} \left.\frac{\partial \mathcal{L}_j}{\partial \theta_k}\right|_{\theta = \mathbf{w}^p} \cdot (w_k^c - w_k^p)} \tag{6}$$

This is the **gene score**: a scalar estimating how much reverting gene i would change the loss on domain j.

- If $\delta_{ij} < 0$: reverting gene i would *decrease* the loss on domain j (repairing would help—the gene was *damaged* by training).
- If $\delta_{ij} > 0$: reverting would *increase* the loss (the gene's changes actually *benefit* domain j).
- If $\delta_{ij} \approx 0$: the gene's changes had negligible effect on domain j.

Key Idea

The crucial efficiency insight: the gradient $\frac{\partial \mathcal{L}_j}{\partial \theta_k}$ for *all* d parameters is computed by a single forward+backward pass (backpropagation). Once we have this gradient vector, we extract each gene's score by slicing out the relevant parameters and computing the dot product with the weight change. Total cost: one forward+backward pass per prior domain, yielding *all* gene scores simultaneously.

Example

Suppose gene i consists of 3 parameters with:

- Gradients: $\frac{\partial \mathcal{L}_j}{\partial \theta_k} = (0.5, -0.3, 0.1)$
- Weight changes: $w_k^c - w_k^p = (-0.2, 0.4, -0.1)$

Then $\delta_{ij} = (0.5)(-0.2) + (-0.3)(0.4) + (0.1)(-0.1) = -0.1 - 0.12 - 0.01 = -0.23$.
Since $\delta_{ij} < 0$, reverting this gene would decrease the loss on domain j by approximately 0.23. This gene was *damaged* by fine-tuning and is a repair candidate.

Sign convention in the code. In the implementation, the raw score is computed as Eq. 6. When assessing whether a gene is deleterious, the scores are *negated*: we define $d_{ij} = -\delta_{ij}$, so that positive d_{ij} means "repairing helps" (the gene is deleterious). The Bayesian analysis is performed on these negated scores.

4 Split-Half Noise Estimation

4.1 Why We Need Noise Estimation

The gene score δ_{ij} from Eq. 6 is computed using a finite set of evaluation samples. If we used a different random subset, we would get a slightly different score. This randomness is **observation noise**.

Some genes might have $\delta_{ij} = -0.23$ purely by chance, even if the true effect is zero. If we repair every gene with a negative score, we would be making many false-positive repairs—reverting genes that weren't actually damaged. We need to estimate how noisy the scores are.

4.2 The Split-Half Procedure

Given N evaluation samples, we split them into two halves (A and B), each of size $N/2$:

$$\delta_{ij}^A = \text{gene score using samples from half A} \tag{7}$$

$$\delta_{ij}^B = \text{gene score using samples from half B} \tag{8}$$

The combined score is the average: $\delta_{ij} = (\delta_{ij}^A + \delta_{ij}^B)/2$.

Estimating noise variance. Each half-score has some noise variance σ^2_{half}. The difference $\delta^A - \delta^B$ has variance $2\sigma^2_{\text{half}}$ (since the halves are independent). The combined average $(\delta^A + \delta^B)/2$ has variance $\sigma^2_{\text{half}}/2$. Therefore:

$$\sigma^2 = \frac{\text{Var}(\delta^A - \delta^B)}{4} \tag{9}$$

where the variance is taken across genes (treating $\delta_i^A - \delta_i^B$ for $i = 1, \ldots, L$ as a sample).

Split-half correlation. As a diagnostic, we also compute the Pearson correlation between δ^A and δ^B across genes. High correlation ($r \approx 1$) indicates the scores are reliable and reproducible; low correlation indicates noise dominance. In our experiments, r ranged from 0.83 to 0.999.

Example

With 49 evaluation samples, we use 24 for half A and 24 for half B (dropping 1). Suppose across 928 genes, we compute δ_i^A and δ_i^B for each. The differences $\delta_i^A - \delta_i^B$ have variance 0.0012. Then $\sigma^2 = 0.0012/4 = 0.0003$, and $\sigma = 0.017$.
If the overall variance of δ_i is 0.005, the signal-to-noise ratio is $(0.005 - 0.0003)/0.0003 \approx 15.7$—plenty of signal.

5 Empirical Bayes Shrinkage

5.1 The Generative Model

We now have, for each gene i and prior domain j:

- An observed score d_{ij} (the negated combined score).
- An estimate of its noise variance σ^2 (from split-half).

We model the scores across genes using the conjugate normal model from Section 1.4:

$$\text{Prior (true gene effects):} \qquad \mu_i \sim \mathcal{N}(\mu_0, \tau^2) \tag{10}$$

$$\text{Likelihood (noisy observation):} \qquad d_i \mid \mu_i \sim \mathcal{N}(\mu_i, \sigma^2) \tag{11}$$

Here μ_i is the *true* effect of reverting gene i (unknown), and d_i is the noisy observed score. The prior says that most genes have effects near μ_0 (the population mean), with spread τ^2.

5.2 Estimating the Parameters (Method of Moments)

The model has three unknown parameters: μ_0, τ^2, and σ^2. We estimate them from data:

Noise variance σ^2: Already estimated from split-half (Eq. 9).

Prior mean μ_0: Estimated as the sample mean of all gene scores:

$$\hat{\mu}_0 = \frac{1}{L} \sum_{i=1}^{L} d_i$$

Signal variance τ^2 (method of moments): The total observed variance across genes is:

$$\mathrm{Var}(d) = \frac{1}{L} \sum_{i=1}^{L} (d_i - \hat{\mu}_0)^2$$

Under our model, $\mathrm{Var}(d) = \tau^2 + \sigma^2$ (the observed spread is the sum of true variation plus noise). Therefore:

$$\hat{\tau}^2 = \max\big(0,\ \mathrm{Var}(d) - \sigma^2\big) \tag{12}$$

The $\max(0, \cdot)$ ensures we don't get a negative variance estimate (which would happen when noise exceeds total variance—meaning there is essentially no signal).

5.3 Computing the Posterior

Using the conjugate formula (Eq. 3):

$$\boxed{B = \frac{\hat{\tau}^2}{\hat{\tau}^2 + \sigma^2}, \qquad \delta_i^* = B \cdot d_i + (1 - B) \cdot \hat{\mu}_0, \qquad \mathrm{Var}_{\mathrm{post}} = B \cdot \sigma^2} \tag{13}$$

Key Idea

The shrinkage factor B is the same for all genes within one objective. It acts as a **global confidence dial**: when the objective's gene scores are mostly signal ($B \approx 1$), we trust individual scores; when mostly noise ($B \approx 0$), every gene's score is pulled toward the mean, preventing false positives.

Worked Example: Shrinkage in Action

Consider scoring 928 genes for the "code" domain. Suppose:

- Observed gene scores: mean $\hat{\mu}_0 = 0.001$, variance $\mathrm{Var}(d) = 0.005$
- Noise variance from split-half: $\sigma^2 = 0.0003$

Signal variance: $\hat{\tau}^2 = 0.005 - 0.0003 = 0.0047$.
Shrinkage: $B = 0.0047/(0.0047 + 0.0003) = 0.94$.
This means we trust the observed scores 94% of the way. For a gene with observed score $d_i = 0.05$:

$$\delta_i^* = 0.94 \times 0.05 + 0.06 \times 0.001 = 0.047$$

Barely any shrinkage—the score is reliable.
Now suppose a different objective where $\mathrm{Var}(d) = 0.0004$ and $\sigma^2 = 0.0003$. Then $\hat{\tau}^2 = 0.0001$ and $B = 0.0001/0.0004 = 0.25$. For the same gene:

$$\delta_i^* = 0.25 \times 0.05 + 0.75 \times 0.001 = 0.013$$

Heavy shrinkage—the score dropped from 0.05 to 0.013 because the objective's scores are too noisy to trust.

5.4 Computing Posterior Probabilities

The posterior distribution for gene i's true effect is:

$$\mu_i \mid d_i \sim \mathcal{N}(\delta_i^*,\ \mathrm{Var}_{\mathrm{post}})$$

The probability that gene i is truly deleterious (i.e., repairing it would help) is the probability that the true effect is positive:

$$\boxed{P(\mathrm{del}_i) = P(\mu_i > 0 \mid d_i) = 1 - \Phi\left(\frac{-\delta_i^*}{\sqrt{\mathrm{Var}_{\mathrm{post}}}}\right)} \tag{14}$$

This is a **calibrated probability**: it accounts for observation noise and corrects for multiple testing across genes.

Example

Continuing from above ($B = 0.94$), with $\delta_i^* = 0.047$ and $\mathrm{Var}_{\mathrm{post}} = 0.94 \times 0.0003 = 0.000282$, so $\sqrt{\mathrm{Var}_{\mathrm{post}}} = 0.0168$:

$$P(\mathrm{del}_i) = 1 - \Phi\left(\frac{-0.047}{0.0168}\right) = 1 - \Phi(-2.80) = 0.997$$

We are 99.7% confident this gene is deleterious—strong evidence for repair.
For the noisy objective ($B = 0.25$), $\delta_i^* = 0.013$, $\mathrm{Var}_{\mathrm{post}} = 0.25 \times 0.0003 = 0.000075$, $\sqrt{\mathrm{Var}_{\mathrm{post}}} = 0.0087$:

$$P(\mathrm{del}_i) = 1 - \Phi(-0.013/0.0087) = 1 - \Phi(-1.49) = 0.932$$

Still fairly confident (93.2%), but less so. The shrinkage appropriately reduced our certainty.

6 The Repair Decision

6.1 Multi-Objective Aggregation

The scoring procedure yields $P(\mathrm{del}_{ij})$ for each gene i and prior domain j, plus $P(\mathrm{ben}_i)$ for the current domain (computed analogously, using the raw—not negated—scores). To make a single repair decision per gene, we aggregate across domains by taking the **maximum** deleterious probability:

$$P_{\max}(i) = \max_{j \in \text{prior domains}} P(\mathrm{del}_{ij})$$

This conservative strategy means a gene is flagged for repair if it damages *any* prior domain, even if it's fine for others.

6.2 The Trade-off Score

Repairing a gene undoes damage to prior domains but may also remove benefit to the current domain. The **trade score** balances these:

$$\mathrm{trade}_i = P_{\max}(i) - \alpha \cdot P(\mathrm{ben}_i) \tag{15}$$

where $\alpha \in [0, 1]$ controls how much we value current-domain benefit relative to prior-domain protection. In our experiments, $\alpha = 0.3$.

6.3 Decision Rules

Repair: Gene i is a repair candidate if $\mathrm{trade}_i > \theta$ (threshold, default $\theta = 0.8$). All candidates are ranked by trade score (highest = most damaged), and at most p% of genes are repaired per cycle ($p = 15$% in our experiments, i.e., 139 out of 928 genes).

Fix: Gene i is fixed (complement updated) if $P(\mathrm{del}_{ij}) < 0.3$ for *all* prior domains j. This means we are confident the gene's changes are harmless to everything learned so far, so we permanently accept them.

Repair Candidate Selection

After training on domain 3 (medical), with 2 prior domains (code, legal):

Gene	P_{code}	P_{legal}	$P_{\max}$	P_{ben}	Decision
attn_h3_L5	.997	.42	.997	.12	$.997 - 0.3(.12) = .96$ **Repair**
mlp_up_L8	.15	.22	.22	.85	$.22 - 0.3(.85) = -.04$ No repair
ln_L2	.08	.11	.11	.35	$P_{\max} < 0.3$: **Fix**

Gene `attn_h3_L5` is strongly deleterious to code (99.7%) and mildly to legal (42%), with low benefit to medical (12%). It gets repaired.
Gene `mlp_up_L8` has low damage but high benefit (85%)—we keep it.
Gene `ln_L2` has very low damage everywhere (< 0.3) so we fix it: copy primary $\rightarrow$ complement, accepting the adaptation permanently.

7 Experimental Results

All experiments use Qwen2.5-7B (7.6B parameters, $L = 928$ genes), fine-tuned for 3 epochs per domain in bfloat16 with an 8-bit Adam optimizer. Evaluation uses 49 samples of 256 tokens per domain.

Table 1: Geometric mean perplexity after sequential training (lower is better).

Domains	**MOLLI**	**LoRA**	**Impr.**	**Time (M / L)**
3 (code $\rightarrow$ legal $\rightarrow$ medical)	**1.60**	2.10	−24%	38 m / 15 m
5 (+ science, finance)	**1.11**	1.45	−23%	113 m / 49 m
6 (+ general)	**1.52**	1.60	−5%	146 m / 62 m

Table 2: Per-domain perplexity after all sequential training phases.

	gen	code	legal	med	sci	fin	**GM**
MOLLI (5d)	1.06	1.25	1.10	1.14	1.06	1.06	**1.11**
LoRA (5d)	1.10	1.63	1.83	2.29	1.10	1.10	1.45
MOLLI (6d)	1.05	1.53	2.53	2.80	1.05	1.05	**1.52**
LoRA (6d)	1.11	1.68	2.81	2.65	1.11	1.11	1.60

Interpretation. At 5 domains, MOLLI retains near-perfect performance on all prior domains (all ≤ 1.25), while LoRA shows the O(1/N) dilution effect: medical degrades to 2.29 and legal to 1.83. At 6 domains, both methods show some forgetting on middle domains (legal, medical), but MOLLI still maintains a lower overall GM. The narrower gap at 6 domains (5% vs 23%) suggests the fixed 15% repair cap becomes a bottleneck when balancing 7 simultaneous objectives.

Scoring reliability. Split-half correlations ranged from $r = 0.83$ (on noisier objectives like the code domain after medical training) to $r = 0.999$ (on the general domain). Shrinkage factors B

ranged from 0.75 (SNR = 3.1, conservative shrinkage) to 0.999 (SNR = 842.5, minimal shrinkage). In all cases, the empirical Bayes procedure appropriately modulated confidence.

Repair statistics. Exactly 139 genes (the 15% cap) were repaired in each cycle, and 0–151 genes were fixed per domain. The cap was saturated in every cycle, suggesting that relaxing or adapting it could further improve results.

8 Summary

The MOLLI algorithm can be summarized in four key ideas:

1. **Dual-strand encoding** provides a healthy reference (the complement) against which to measure fine-tuning damage.
2. **Gradient-based gene scoring** (Eq. 6) efficiently estimates the effect of reverting each gene using a first-order Taylor approximation, requiring only one forward+backward pass per objective.
3. **Empirical Bayes shrinkage** (Eqs. 12–13) separates signal from noise and produces calibrated posterior probabilities, preventing false-positive repairs.
4. **Selective bidirectional repair** (Eq. 15) undoes damage while preserving beneficial changes, mirroring the biological gene conversion mechanism.

The net effect is a continual learning system that actively maintains prior knowledge rather than passively hoping it survives, achieving 23–24% lower perplexity than LoRA across 3–5 sequential domains.